

Edge AI & TinyML Lab Manual

CSE 521: Embedded Machine Learning

Omar Tahon

ID: 022025001

May 16, 2026

Abstract

This lab manual introduces the concepts, models, and workflows associated with Edge Artificial Intelligence (Edge AI) and TinyML. We explore state-of-the-art architectures optimized for mobile and edge deployments, such as YOLOv11 and MobileNetV4, and delve into small language models (SLMs) like Phi-3. Furthermore, we investigate model compression via quantization and provide practical, self-contained experiments demonstrating how to quantize models and deploy them using tools such as the Texas Instruments `edgeai-tensorlab` ecosystem.

Contents

1 Introduction to Edge AI & TinyML

Edge AI involves running machine learning algorithms locally on hardware devices (the "edge" of the network) rather than relying on centralized cloud servers. This paradigm shift addresses several critical challenges in modern computing:

- **Latency:** Processing data locally ensures real-time responses, crucial for autonomous driving and robotics.
- **Bandwidth:** Sending raw data (like high-res video) to the cloud is expensive; edge AI extracts insights locally.
- **Privacy & Security:** Sensitive user data does not need to leave the device.
- **Power Efficiency:** TinyML specifically focuses on deploying models on ultra-low power microcontrollers (MCUs) consuming milliwatts of power.

2 State-of-the-Art Edge Models

2.1 Vision Models: YOLOv11 and MobileNetV4

Recent advancements have drastically reduced the size and computational requirements of vision models without significantly sacrificing accuracy.

YOLOv11 (November 2024): The latest iteration of the YOLO family introduces a transformer-based backbone with Partial Self-Attention (PSA) and NMS-free training. This results in a 25-40% lower latency compared to its predecessors while achieving 60+ FPS processing on edge devices.

MobileNetV4: Developed by Google, MobileNetV4 introduces the Universal Inverted Bottleneck (UIB) block and Mobile MQA attention. It achieves 87% ImageNet accuracy with merely 3.8ms latency on mobile edge TPUs, making it a universal efficient architecture for the mobile ecosystem.

2.2 Small Language Models (SLMs)

Large Language Models (LLMs) like GPT-4 are too massive for edge deployment. Instead, the industry is pivoting towards highly optimized SLMs:

- **Microsoft Phi-3:** A 3.8B parameter model optimized for CPU and mobile inference, supporting up to 128K context windows.
- **TinyLlama:** A 1.1B parameter model with a highly compact architecture, designed specifically for low-resource environments.

3 Model Compression and Quantization

To fit these models onto constrained hardware, compression techniques are mandatory. Quantization maps high-precision floating-point numbers (FP32) to lower-precision representations (INT8, INT4).

3.1 Quantization Techniques

- **Post-Training Quantization (PTQ):** The model is trained in FP32, and weights/activations are mapped to INT8 post-training. Simple but can degrade accuracy on complex tasks.
- **Quantization-Aware Training (QAT):** The model simulates low-precision behavior during training, learning to compensate for quantization noise, preserving accuracy.
- **Advanced Techniques (AWQ, GPTQ):** Activation-aware Weight Quantization (AWQ) protects salient (critical) weights from quantization, maintaining high accuracy even at 4-bit precision.

4 Experiment 1: Model Quantization

This experiment demonstrates how to simulate quantization on a randomly generated weight matrix using standard numerical libraries. It mirrors the fundamental operations happening inside inference engines.

4.1 Code Implementation

```
1 import numpy as np
2
3 def quantize_int8(weights):
4     # Determine the dynamic range
5     min_val = np.min(weights)
6     max_val = np.max(weights)
7
8     # Calculate scale and zero-point
9     # INT8 range is [-128, 127]
10    scale = (max_val - min_val) / 255.0
11    zero_point = -128 - np.round(min_val / scale)
12
13    # Quantize
14    quantized_weights = np.round(weights / scale + zero_point)
15    # Clip to valid INT8 range
16    quantized_weights = np.clip(quantized_weights, -128, 127).astype(np.int8)
17
18    return quantized_weights, scale, zero_point
19
20 def dequantize(quantized_weights, scale, zero_point):
21     # Dequantize back to float32 for inference/computation
22     return (quantized_weights.astype(np.float32) - zero_point) * scale
23
24 # 1. Generate dummy float32 weights
25 weights_fp32 = np.random.randn(10, 10).astype(np.float32) * 5
26
27 # 2. Quantize
28 weights_int8, scale, zp = quantize_int8(weights_fp32)
29
30 # 3. Analyze Compression Ratio
```

```

31 fp32_size = weights_fp32.nbytes
32 int8_size = weights_int8.nbytes
33 print(f"Compression Ratio: {fp32_size / int8_size}x") # Expected: 4.0x

```

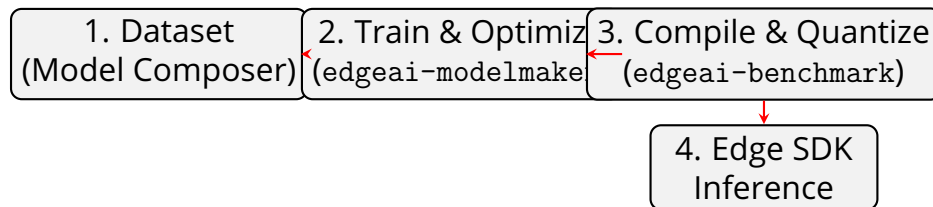
Listing 1: Simulating INT8 Quantization in Python

5 Experiment 2: End-to-End Edge Vision Deployment

In this experiment, we explore the conceptual workflow of deploying a model using the **Texas Instruments** `edgeai-tensorlab` toolchain.

5.1 The `edgeai-tensorlab` Workflow

The TI ecosystem provides an end-to-end pipeline:



5.2 Steps for Deployment

1. **Data Collection:** Use tools like EDGE-AI-STUDIO to capture and annotate images for a specific task (e.g., Object Detection).
2. **Training via Bring Your Own Data (BYOD):** Use `edgeai-modelmaker` (a command-line tool wrapping around architectures like torchvision or mmdetection) to train a model. Model maker automatically handles architecture selection and hyperparameter tuning suitable for Edge processing.
3. **Compilation and Benchmarking:** Pass the trained model to `edgeai-benchmark` or `edgeai-tidlrunner`. This step performs Post-Training Quantization (converting the model to fixed-point integers) and compiles the execution graph into machine code optimized for TI's neural processing units (NPU).
4. **Deployment:** The resulting compiled artifacts are deployed directly to the Edge AI SDK, executing inference pipelines via GStreamer plugins on the physical board.

6 Conclusion

Edge AI is transforming how we deploy intelligence in the real world. By leveraging highly efficient model architectures (YOLOv11, MobileNetV4, Phi-3) and rigorous quantization strategies, modern engineering teams can extract real-time insights from sensor data without the latency, cost, and privacy concerns of cloud computing. Toolchains like `edgeai-tensorlab` abstract away the immense complexity of hardware compilation, allowing developers to focus on data and problem-solving.